

SEMANTIC RAG ENGINE

Similarity metric selection and a production migration path from local FAISS retrieval to Vertex AI Vector Search (Matching Engine).

BENCHMARK SIGNAL

The current benchmark already proves the central design point: better semantic intent representation materially improves retrieval quality without changing the corpus or top-k policy.

RAW SEARCH

0.417

avg recall@k

ENHANCED SEARCH

0.833

avg recall@k

RAW SEARCH

0.417

avg MRR

ENHANCED SEARCH

0.875

avg MRR

DECISION

COSINE SIMILARITY IS THE RIGHT METRIC FOR THIS EMBEDDING SPACE

The retrieval engine uses sentence-transformer vectors where semantic meaning is carried primarily by direction, not magnitude.

Why the metric choice matters

- Transformer embeddings are typically normalized before search, which makes cosine similarity a direct measure of semantic alignment.
- Cosine stays stable when vector magnitude drifts because of phrasing length, token density, or model-specific scaling behavior.
- FAISS inner-product search over normalized vectors is operationally simple and maps cleanly to managed vector search backends.

Cosine similarity

Compares angle between vectors.

Best for dense semantic embeddings where direction captures meaning.

Operational fit:
normalize once, search consistently, rank semantically.

Euclidean distance

Compares absolute geometric distance.

More sensitive to magnitude and scaling variance.

Operational risk:
can over-penalize vectors that are semantically close but numerically longer.

Semantic signal	Angle of alignment	Absolute distance in space
Sensitivity to scale	Low after normalization	Higher
FAISS implementation	IndexFlatIP on normalized vectors	IndexFlatL2
Recommendation	Preferred	Secondary only for non-normalized features

EVIDENCE

THE BENCHMARK SHOWS THAT BETTER QUERY SEMANTICS DRIVE BETTER RETRIEVAL

Both strategies use the same corpus, chunking policy, vector index, and similarity metric. The gain comes from richer query representation before embedding.

STRATEGY A

0.417

Average recall@k

STRATEGY B

0.833

Average recall@k

STRATEGY A

0.417

Average MRR

STRATEGY B

0.875

Average MRR

What the score change actually means

Strategy B nearly doubles average recall because the rewritten query lands closer to the engineering language present in the indexed corpus.

MRR also improves sharply, which means the first relevant chunk moves materially closer to rank one. This is the behavior expected from a retrieval system whose semantic representation is improving without changing the index itself.

Why Euclidean would be weaker here

If search ranking depended more on vector magnitude, these gains would be noisier because longer or denser rewrites could distort distance. Cosine avoids that failure mode and preserves the architectural intent: compare meaning, not scale.

Interpretation: cosine similarity remains constant across both strategies, so the benchmark isolates the effect of semantic query expansion. That is the correct design behavior for a RAG retrieval tier.

TARGET STATE

VERTEX AI MATCHING ENGINE IS AN EVOLUTION OF THE RETRIEVAL TIER, NOT A REDESIGN

The application contracts around chunking, embeddings, query expansion, metadata, and benchmark evaluation should stay stable while the vector backend becomes managed and production-grade.

1. Ingestion

Document loader
Preprocessing
Chunking
Metadata capture

2. Embeddings

SentenceTransformer
today
Vertex-hosted or
managed model later
Versioned embedding
contract

3. Vector Index

FAISS local index today
Matching Engine
deployed index in
production
Approximate nearest
neighbor at scale

4. Retrieval API

Query rewrite
Embedding lookup
Top-k fetch

5. Response

Ranked chunks
Scores
Observations

What stays stable

VectorStore remains the architectural seam. The application still owns chunk identity, metadata shape, benchmark logic, and the retrieval API contract.

This is important because productionizing the vector layer should not force a rewrite of the semantic retrieval service.

What changes in production

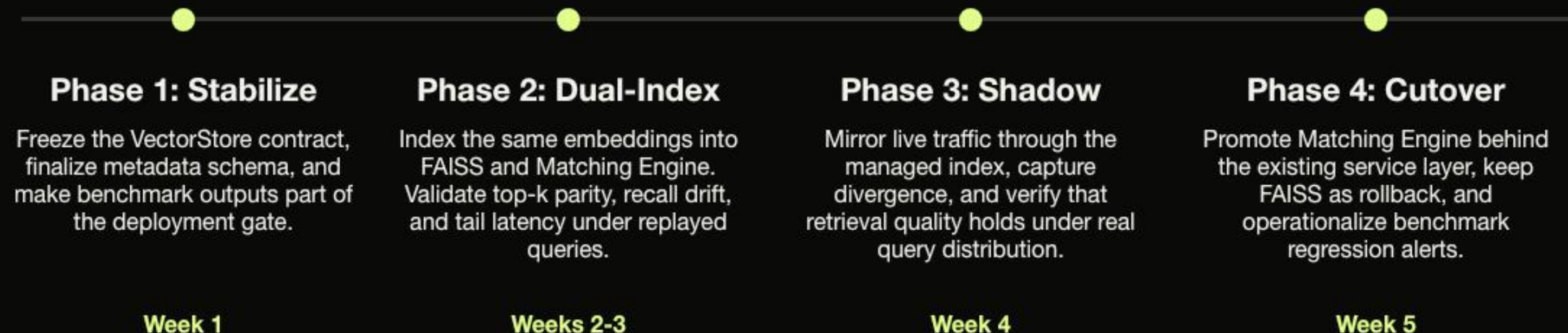
Index build moves out of process, vector search becomes managed, metadata shifts to a persistent store, and deployment adds observability, IAM, versioned rollouts, and latency budgets.

The migration is therefore operational, not conceptual.

ROADMAP

MIGRATION SHOULD BE PHASED, BENCHMARK-GATED, AND REVERSIBLE

The correct production move is not a hard swap from FAISS to Matching Engine. It is a controlled rollout with interface hardening, parity checks, shadow traffic, and a retained rollback path.



Recommended end state: Matching Engine for managed vector search, persistent metadata storage, automated benchmark checks, and the same query-expansion-led retrieval contract retained at the application layer.